

LOW LEVEL DESIGN (LLD)

1. Introduction

This document describes the internal implementation details and logic of the system. It focuses on how each module operates and interacts with others.

2. Module Breakdown

Ingest.py

Responsible for:

- Loading PDF using loader
- Splitting text into chunks (500 size, 100 overlap)
- Generating embeddings
- Storing embeddings in ChromaDB

retriever.py

- Connects to vector database
- Performs similarity search

- Returns top matching chunks
- `groq_llm.py`
- Sends prompt + retrieved context to Groq API
- Uses llama-3.1-8b-instant
- Receives generated response

`graph.py`

- Defines nodes and edges in LangGraph
- Handles routing logic
- Detects sensitive queries

app.py

- Entry point of application
- Accepts user input
- Calls retriever and LLM
- Displays output

3. Detailed Workflow Steps:

- ❖ Load and preprocess PDF
- ❖ Split into chunks
- ❖ Generate embeddings
- ❖ Store in DB
- ❖ User enters query

❖ Convert query to embedding

❖ Retrieve similar chunks

❖ Pass to LLM

❖ Generate response

❖ Check for escalation

4. Algorithm

❖ Initialize system

❖ Load knowledge base

❖ Split text into chunks

❖ Generate embeddings

❖ Store embeddings in database

Loop:

Accept user query

Convert query to embedding

Retrieve relevant chunks

Pass chunks to LLM

If query contains sensitive keywords:

Escalate to human

—

5. Data Handling

- ❖ Text chunks stored as documents
- ❖ Embeddings stored as vectors
- ❖ Query processed dynamically

6. Error Handling

- ❖ Handles empty retrieval
- ❖ Handles API failure
- ❖ Handles invalid inputs

